B.TECH BCSE497J - PROJECT 1

Authoritative Implementation and Research Specification

Active Visual Discovery of GUI Action Preconditions

System: Vision GUI Agent

Version: 1.0 - Approved implementation baseline

Effective date: 29 August 2026

Team: Mantri Vishnu Vikranth; Harish Kumar V S; Abhinav Pant

Faculty guide: Dr. Sendhil Kumar K S

Authority: Single source of truth for implementation, evaluation, and paper claims

IMPLEMENTATION DECISION

> The current screenshot-native state graph becomes the baseline. The research contribution is active, evidence-backed discovery of observable GUI action preconditions through safe contrasting interventions.

This specification supersedes earlier implementation plans where they conflict with it.

Contents

1. Authority and Change Control
2. Executive Decision
3. Research Position
4. Why This Work Is Needed
5. Project Scope
6. Terminology and Formal Model
7. System Architecture
8. Required Processing Pipeline
9. Implementation Plan Mapped to the Current Repository
10. Controlled Benchmark Specification
11. Baselines and Ablations
12. Metrics and Analysis
13. Safety Requirements
14. Verification and Testing Requirements
15. Development Phases
16. How the Proposed Method Is Better
17. Risks and Mitigations
18. Required Deliverables
19. Paper Blueprint
20. Traceability to Review 1
21. References
22. Final Implementation Directive

1. Authority and Change Control

This document is the single source of truth for the next implementation and evaluation phase of the Vision GUI Agent project. All code changes, experiments, documentation, demonstrations, and paper claims MUST conform to this specification.

When another project document conflicts with this specification, this specification takes precedence. The July 2026 Review 1 presentation remains a historical record of the original motivation, not the current novelty claim or implementation plan.

Changes to the following items require a versioned amendment to this document before implementation:

- the primary research question;
- the definition of the proposed method;
- the pixel-only information boundary;
- the benchmark and baseline definitions;
- the primary metrics;
- the safety boundary for autonomous experiments;
- the minimum deliverables.

Routine engineering decisions that do not alter those items MAY be recorded in code comments, issues, or an architecture decision record. Results MUST never be edited into the hypotheses after experiments have begun.

Normative terms are used deliberately:

- **MUST / MUST NOT:** mandatory for project compliance;
- **SHOULD / SHOULD NOT:** expected unless a documented reason justifies a deviation;
- **MAY:** optional.

2. Executive Decision

The project will continue to build a screenshot-native agent that interacts through mouse and keyboard input without using application APIs, the DOM, accessibility trees, selectors, source code, or hidden application state.

The project will **not** claim that vision-based GUI automation, persistent state graphs, autonomous GUI exploration, transition memory, or immediate action-effect descriptions are new. Recent work already covers those areas.

The research contribution will instead investigate whether a vision-only agent can actively discover **hidden action preconditions** through controlled, contrasting GUI interactions and store the resulting knowledge as explicit, evidence-backed action schemas.

The system will progress from memory of topology:

```
Screen A -- click Export --> Screen B
```

to an explicit action model:

```
Capability: export_document
Precondition: document_open
Action: activate the visible Export control
Effect: export_dialog_visible
Evidence: successful and unsuccessful trials in contrasting states
Confidence: evidence-derived score
```

The current state graph remains valuable. It becomes a baseline and a navigation aid rather than the paper's novelty.

3. Research Position

3.1 Primary research question

> *Can a screenshot-only GUI agent actively discover the observable preconditions governing GUI actions and use the resulting evidence-backed action model to solve unseen tasks more effectively than stateless execution, trajectory replay, and UI state-transition graph memory?*

3.2 Primary hypothesis

H1 - Active precondition learning: An agent using intervention-validated precondition/effect schemas will make fewer invalid action attempts and achieve higher success on unseen conditional tasks than the same agent using only a persistent UI state graph.

3.3 Secondary hypotheses

- **H2 - Intervention value:** Active contrasting experiments will produce more accurate precondition models than passive inference from successful before/after transitions alone.
- **H3 - Compositional reuse:** Explicit schemas will support unseen goals that require composing previously learned capabilities, even when no identical completed trajectory exists.
- **H4 - Layout robustness:** Semantic predicates and action schemas will retain more utility under controlled layout changes than screenshot-identity or coordinate-based memory.
- **H5 - Knowledge efficiency:** A compact action model will preserve useful task-solving knowledge with fewer stored units than raw screenshot trajectories for the evaluated task set.

3.4 Falsifiability

The research remains valid if one or more hypotheses are rejected. A negative result MUST be reported rather than hidden. The paper's contribution is the method, benchmark, and controlled evidence, not a precommitted claim of superiority.

3.5 Defensible novelty statement

The project MAY use the following qualified claim after the implementation and literature review are complete:

> *To the best of our knowledge, we present and evaluate a screenshot-only method that induces explicit GUI action schemas with visually grounded preconditions, effects, uncertainty, and supporting evidence through safe contrasting interventions, then uses those schemas for compositional planning on unseen tasks.*

The project MUST NOT use claims such as "the first cause-and-effect GUI memory," "the first visual GUI graph," or "no existing agent remembers action effects."

4. Why This Work Is Needed

The Review 1 presentation correctly identified a practical need: many desktop, legacy, remote, canvas-rendered, and closed-source interfaces cannot be relied upon to expose stable APIs, DOM structures, selectors, or accessibility identifiers. A screenshot-and-input boundary is therefore still useful.

Its original novelty argument, however, is no longer defensible. GUI-explorer mines operational action-effect descriptions from observation-action-outcome triples; graph-memory systems store executable state transitions; world models forecast post-action states; and verification-driven agents reason about expected immediate effects. Earlier GUI-testing research also represented actions with preconditions and effects, although those models were typically supplied by designers rather than learned from pixels.

The remaining problem is not simply remembering that an action changed the screen. It is learning **when** a semantic operation is applicable and gathering evidence strong enough to distinguish a genuine enabling condition from an incidental visual correlation.

For example, observing one successful export does not establish that an open document is required. The agent must compare outcomes across states in which candidate conditions differ. This is the role of active intervention.

5. Project Scope

5.1 In scope

The implementation MUST include:

1. Screenshot-only perception and semantic element grounding.
2. Browser and visible-desktop input adapters using pixel-coordinate mouse and keyboard operations.
3. Persistent UI state-transition graph memory as the existing baseline.
4. Extraction of visually grounded semantic state predicates.
5. Persistent semantic action schemas.
6. Passive effect hypothesis generation from before/action/after transitions.
7. Precondition hypothesis generation from successes, ineffective actions, and contrasting states.
8. Safe active experiment selection for uncertain preconditions.
9. Evidence and confidence tracking, including contradictions.
10. Action-model planning that can establish unmet preconditions before attempting a goal action.
11. Visual verification of predicted effects.
12. A controlled benchmark with hidden evaluator state and deterministic reset.
13. Baseline, ablation, and statistical evaluation.
14. Reproducible artifacts, logs, and a final research report.

5.2 Out of scope for Version 1.0

The following MUST NOT become required deliverables:

- universal causal discovery for arbitrary software;
- unrestricted autonomous exploration of real user applications;
- interventions involving purchases, submissions, messages, deletion, account changes, deployment, or other irreversible operations;
- learning from DOM, accessibility, application APIs, source code, manifest files, OCR APIs tied to application structure, or hidden benchmark state;
- cross-application transfer as a primary hypothesis;
- reinforcement learning or fine-tuning a new foundation model;
- replacing the visual grounder or base LLM with a custom trained model;

- full PDDL support, probabilistic programming, or a general-purpose theorem prover;
- support for every operating system;
- claiming that all software state is observable from pixels.

Cross-application transfer MAY be included as a small exploratory analysis only after all required experiments are complete.

5.3 Experimental scope

Version 1.0 is limited to observable Boolean and small categorical conditions, deterministic or near-deterministic actions, and immediate or short-horizon effects that can be verified visually within a bounded number of observations.

Examples include:

- `document_open`;
- `form_complete`;
- `authenticated`;
- `item_selected`;
- `advanced_mode_enabled`;
- `export_control_enabled`;
- `dialog_visible`;
- `save_status = saved | unsaved`.

Latent conditions that cannot be distinguished visually MUST be marked `unobservable` or `unknown`; the agent MUST NOT invent evidence for them.

6. Terminology and Formal Model

6.1 Observation

An observation O_t is the raw screenshot captured at time t, its numbered visual annotation, the set of visually detected elements, and a semantic screen label. URL, document title, DOM data, accessibility data, and application metadata are excluded from the agent-facing observation.

6.2 Visual predicate

A visual predicate is a normalized claim about the current observable GUI state with explicit grounding evidence.

```
predicate:
  name: document_open
  value: true
  confidence: 0.96
  grounding:
    - source: visible_text
      value: "Quarterly Report"
      element_signature: "heading|text|quarterly report"
```

A predicate MUST include:

- a stable semantic name;
- a typed value;
- confidence;
- visual grounding or an explicit unobservable status;
- the observation identifier from which it was derived.

6.3 Semantic action

A semantic action is a normalized operation independent of transient element numbers or coordinates, for example open_document, select_item, enable_advanced_mode, or export_document.

The executable action remains grounded at runtime to a currently visible actionable element.

6.4 Transition

A transition is:

```
T_t = (O_t, P_t, A_t, O_t+1, P_t+1, outcome, verification)
```

where P_t and P_t+1 are predicate sets before and after action A_t.

6.5 Effect

An effect is a consistent predicate change associated with a semantic action:

```
positive effects = P_t+1 - P_t
negative effects = P_t - P_t+1
```

An observed delta is initially a hypothesis. It becomes a validated effect only after repeated support and no unresolved material contradiction under the declared scope.

6.6 Precondition

A precondition is a predicate condition whose satisfaction is required, or strongly supported as required, for an action to produce its intended effect.

Precondition status MUST be one of:

- required;
- not_required;
- conditional;
- unknown;
- unobservable.

6.7 Intervention

An intervention is an intentionally selected, safe interaction sequence that changes one or a small number of candidate predicates before retesting a semantic action. Its purpose is to distinguish competing precondition hypotheses.

6.8 Action schema

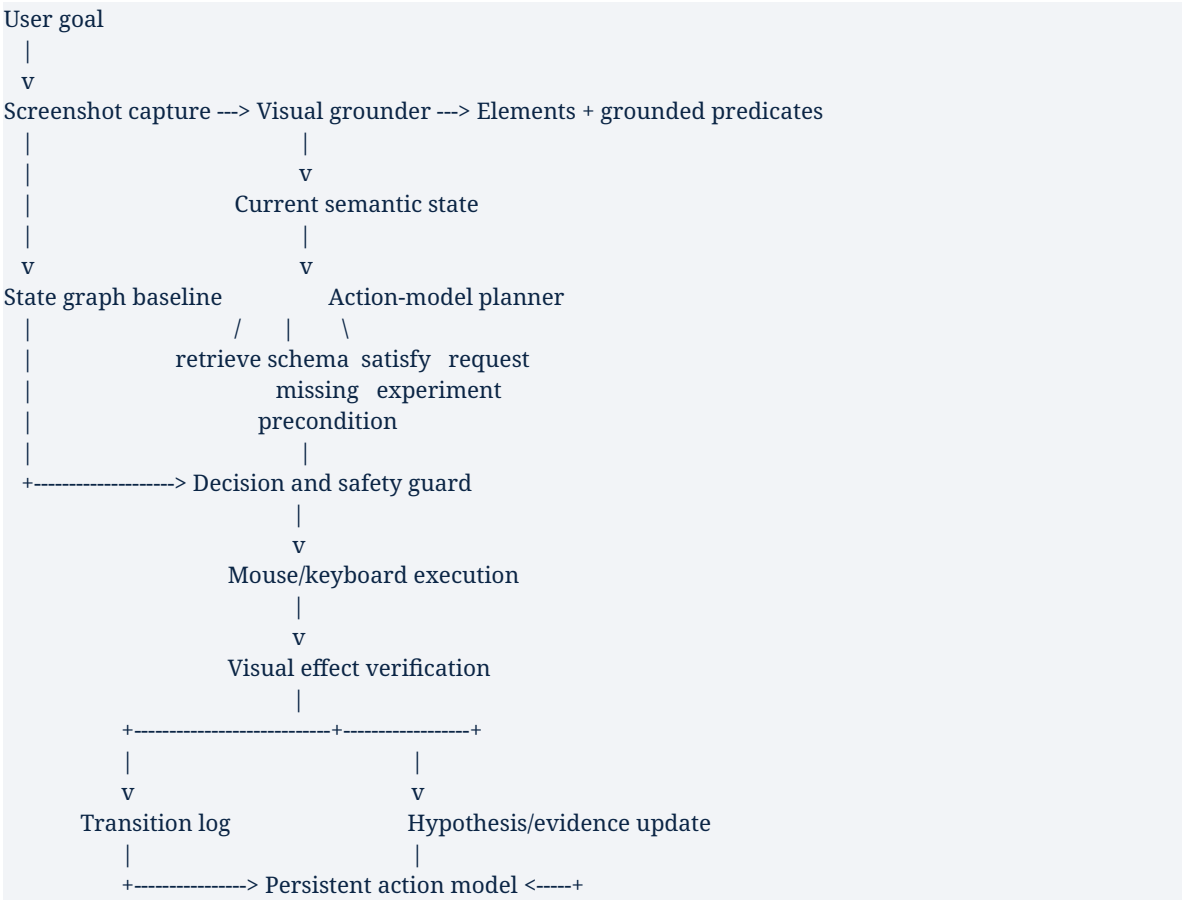
The core learned unit is:

```
action_schema:
 id: export_document.v1
 semantic_name: export_document
 scope: controlled_document_app
 parameters:
  format: [pdf, png]
 target_signature:
  role: button
  semantic_label: export
 preconditions:
  - predicate: document_open
    required_value: true
    status: required
    confidence: 0.91
 effects:
```

```
  - predicate: export_dialog_visible
    resulting_value: true
    confidence: 0.94
safety_class: harmless_reversible
evidence_ids: [transition-17, transition-24, experiment-8]
contradictions: []
version: 1
```

Schemas MUST be human-inspectable and serializable.

7. System Architecture



The current graph and the proposed action model coexist:

- the graph remembers observed navigation topology and supports the existing replay baseline;
- the action model stores semantic conditions and effects that can be reused compositionally;
- the planner MAY use graph paths for navigation while using schemas for functional reasoning;
- evaluation MUST be able to disable each memory component independently.

8. Required Processing Pipeline

8.1 Observe and ground

The system MUST:

1. capture a screenshot;
2. identify actionable controls and state evidence from pixels;
3. reject invalid or out-of-bounds regions;
4. create position-independent element signatures;
5. extract a bounded set of task-relevant predicates;
6. retain the exact visual evidence supporting each predicate.

The agent MUST NOT treat its own natural-language inference as verified state unless the claim is grounded in current visual evidence.

8.2 Normalize predicates

Predicate names MUST use a controlled vocabulary within each benchmark domain. Equivalent surface forms SHOULD normalize to the same predicate:

```
"Document: Budget.docx"
"Editing Budget"
"Budget.docx - Editor"
    -> document_open = true
```

Layout position MUST NOT be part of predicate identity. Content-specific values SHOULD be separated from functional predicates so that changing a username or document name does not create a new functional state.

8.3 Classify outcomes

Every attempted action MUST receive exactly one outcome class:

- `effective`: intended verified effect occurred;
- `ineffective`: action executed but the intended effect did not occur;
- `execution_error`: input could not be delivered or observation failed;
- `ambiguous`: evidence is insufficient to classify;
- `unsafe_skipped`: the safety guard prohibited the action.

`execution_error` and `ambiguous` observations MUST NOT be used as negative causal evidence.

8.4 Generate effect hypotheses

For an effective transition, the learner MUST compare grounded predicate sets before and after the action. Stable deltas become candidate effects.

Effects SHOULD be generalized across content values and layouts. A single transition MAY create a candidate effect but MUST NOT establish a high-confidence rule.

8.5 Generate precondition hypotheses

For repeated instances of the same semantic action, the learner MUST compare:

- predicates present in successful trials;
- predicates absent in successful trials;
- predicates present or absent in ineffective trials;
- known confounding predicates;
- the intended effect being tested.

A predicate is evidence for a required precondition when it is consistently present for successful effect production and its controlled absence is associated with failure. A successful trial while that predicate is absent is direct contradictory evidence against necessity.

The learner MUST distinguish:

- an action being unavailable;
- an action being visible but disabled;
- an action executing without the intended effect;
- an action producing a different conditional effect.

8.6 Select an intervention

When material precondition hypotheses remain uncertain, the experiment selector SHOULD choose a safe reachable state that best distinguishes the candidates while minimizing interaction cost and risk.

The Version 1.0 selection rule is deliberately simple:

1. enumerate candidate predicates whose status is unknown or conditional;
2. enumerate safe, known actions capable of changing one candidate predicate;
3. prefer experiments that vary the fewest other predicates;
4. prefer reversible actions and states with deterministic reset;
5. reject experiments outside the remaining action or risk budget;
6. retest the target semantic action and verify its intended effect.

A complex learned experiment policy is not required.

8.7 Update confidence and contradictions

Each precondition and effect MUST track supporting and contradicting evidence separately. Confidence MUST be computed from evidence, not copied from an LLM response.

Version 1.0 SHOULD use a transparent smoothed ratio:

```
confidence = (support + 1) / (support + contradiction + 2)
```

Evidence weights MAY differ only if the weighting rule is fixed before evaluation. Controlled intervention evidence SHOULD be reported separately from passive evidence.

No schema is permanently final. New contradictory evidence MUST lower confidence or change the rule status.

8.8 Plan with the action model

The action-model planner MUST support bounded backward chaining:

1. parse the goal into desired observable predicates;
2. retrieve schemas whose effects can establish those predicates;
3. check the schemas' required preconditions against the current grounded state;
4. recursively select safe schemas that establish missing preconditions;
5. produce a bounded sequence;
6. execute one action at a time with fresh grounding and effect verification;
7. replan when an expected effect fails.

Search MUST be bounded by maximum depth, action count, and confidence threshold. When no trustworthy schema applies, the system MAY fall back to the existing VLM policy.

8.9 Verify effects

Every schema-selected action MUST declare an observable expected effect. The verifier MUST test that effect from a fresh screenshot. A graph-node change by itself is insufficient when a more specific predicate effect is available.

8.10 Persist knowledge

The latest action model MUST be exported atomically to:

```
artifacts/action-model.json
```

Normalized evidence and experiment records MUST be stored in the existing SQLite artifact database. Exported JSON MUST contain only learned schemas and evidence references, not secret credentials or uncontrolled full model prompts.

9. Implementation Plan Mapped to the Current Repository

9.1 Components to retain

The following existing components remain part of the architecture:

- vision_gui_agent/perception.py: screenshot-native element grounding;
- vision_gui_agent/executor.py: pixel-coordinate input execution;
- vision_gui_agent/verification.py: post-action visual verification;
- vision_gui_agent/state_graph.py: persistent state-graph baseline;
- vision_gui_agent/decision.py: fallback VLM policy and structured decision parsing;
- vision_gui_agent/agent.py: main observe-decide-act loop and safety guard;
- vision_gui_agent/logging_store.py: run and transition evidence;
- vision_gui_agent/desktop.py: visible-desktop adapter;
- vision_gui_agent/models.py: shared typed data structures;
- tests/test_core.py: regression foundation.

The existing test suite MUST continue to pass throughout development.

9.2 New modules

The implementation SHOULD add the following focused modules:

vision_gui_agent/predicates.py

Responsibilities:

- define predicate normalization;
- extract grounded predicates from observations;
- compare predicate states;
- reject ungrounded claims;
- generate stable predicate signatures.

vision_gui_agent/action_model.py

Responsibilities:

- store and version action schemas;
- ingest classified transitions;
- propose and update effects;

- propose and update preconditions;
- retain support and contradiction evidence;
- retrieve schemas by goal effect and current state;
- import/export action-model.json atomically.

vision_gui_agent/experimentation.py

Responsibilities:

- enumerate unresolved hypotheses;
- identify safe discriminating interventions;
- enforce experiment and risk budgets;
- record intervention plans and results;
- prohibit high-impact experimentation.

vision_gui_agent/functional_planner.py

Responsibilities:

- convert goal predicates into bounded schema plans;
- recursively establish missing preconditions;
- rank plans by confidence, reliability, action count, and risk;
- fall back cleanly when no valid plan exists.

These modules MUST remain separable so the experiment can run with the action model or intervention selector disabled.

9.3 Extensions to existing models

models.py MUST define typed records for at least:

- VisualPredicate;
- PredicateGrounding;
- SemanticAction;
- ActionEffect;
- ActionPrecondition;
- ActionSchema;
- HypothesisEvidence;
- ExperimentPlan;

- ExperimentResult.

All model records MUST support strict validation and deterministic serialization.

9.4 Logging changes

The SQLite store MUST retain:

- before and after predicate JSON;
- semantic action identity;
- intended effect;
- outcome class;
- schema identifier, when used;
- whether the decision came from stateless policy, graph replay, passive schema, or active schema planning;
- experiment identifier;
- support or contradiction classification;
- model token usage when available;
- observation, inference, execution, verification, and persistence latency.

Database migration MUST preserve existing run data.

9.5 Configuration and CLI

The system MUST expose comparable modes:

```
--memory-mode none
--memory-mode graph
--memory-mode passive-action-model
--memory-mode active-action-model
```

It SHOULD additionally expose:

```
--experiment-budget <integer>
--min-schema-confidence <0..1>
--max-plan-depth <integer>
--benchmark-reset <configured reset strategy>
```

Active experimentation MUST be disabled by default outside an explicitly configured sandbox benchmark.

10. Controlled Benchmark Specification

10.1 Purpose

The benchmark must reveal whether an agent can learn hidden functional conditions. Ordinary navigation tasks are insufficient because a state graph may solve them without explicit precondition reasoning.

10.2 Environment

Implement a deterministic, locally hosted benchmark named **Visual Function Lab**. It MAY be rendered in a browser for implementation convenience, but the agent MUST receive only screenshots and send only mouse/keyboard input. The benchmark evaluator MAY access hidden state solely for reset and scoring.

The benchmark MUST provide:

- deterministic reset to named initial states;
- at least three visual layouts or themes with unchanged functionality;
- randomized content values that do not alter functional rules;
- hidden ground-truth predicates and action schemas;
- action and state instrumentation unavailable to the agent;
- safe, reversible interactions only;
- a final-state evaluator independent of the agent's self-reported completion.

10.3 Functional domains

The minimum benchmark SHOULD contain three domains:

Document workspace

- open/close document;
- edit content;
- save document;
- export document;
- choose export format.

Example hidden rules:

- export requires a document to be open;
- save requires a document and unsaved changes;

- format selection is available only after the export dialog opens.

Data workspace

- load dataset;
- select a row;
- apply transformation;
- generate report.

Example hidden rules:

- transformation requires loaded data;
- row action requires a selected row;
- report generation requires at least one applied transformation.

Account/settings workspace

- authenticate;
- enable advanced mode;
- edit an advanced setting;
- apply configuration.

Example hidden rules:

- advanced controls require advanced mode;
- applying account changes requires authentication and a complete form.

10.4 Minimum knowledge content

The controlled benchmark MUST include at least:

- 12 semantic actions;
- 8 required-precondition relations;
- 8 immediate observable effects;
- 4 actions that are visible but ineffective or disabled when a precondition is absent;
- 3 conditional effects;
- 3 distractor predicates correlated in training states but not causally required;
- 3 layouts for robustness evaluation.

10.5 Task sets

Tasks MUST be divided before final experiments into:

- **exploration tasks:** interactions from which knowledge may be learned;
- **development tasks:** used while implementing and tuning;
- **held-out unseen tasks:** never used to form identical completed trajectories;
- **layout-shift tasks:** same functional rules under a new visual arrangement;
- **composition tasks:** require at least two schemas whose exact combined trajectory was not observed during exploration.

The split MUST be stored in version control before final evaluation.

10.6 Real-application case study

After controlled evaluation, the system SHOULD be demonstrated on at least one real desktop application, preferably a document editor such as LibreOffice Writer, inside a disposable environment.

The case study is evidence of practical applicability, not ground-truth causal accuracy. It MUST NOT replace the controlled benchmark.

11. Baselines and Ablations

All systems MUST use the same visual grounder, base policy model, task inputs, action budget, retry limit, and environment reset conditions unless the changed component is the subject of an ablation.

B0 - Stateless visual agent

Current screenshot, goal, and bounded recent history only. No persistent graph or action model.

B1 - Trajectory replay

Retrieve an identical or semantically matched completed workflow without graph reasoning.

B2 - Current state graph

Use the current persistent UI state-transition graph and reliability-weighted replay.

B3 - Passive action-effect memory

Store immediate operational descriptions derived from before/action/after observations, without explicit learned preconditions or active experiments. This is the closest internal approximation to transition-aware action-effect memory.

B4 - Passive explicit action model

Infer explicit preconditions and effects from available transitions but do not choose new experiments to resolve uncertainty.

P - Active action model

Full proposed system with explicit schemas, hypothesis evidence, safe contrasting interventions, and schema-based planning.

Required ablations

- P without intervention selection;
- P without contradiction tracking;
- P without graph navigation;
- P with effects but preconditions hidden from the planner;
- P under layout shift.

12. Metrics and Analysis

12.1 Primary metrics

1. **Held-out task success:** evaluator-confirmed completed tasks divided by attempted tasks.
2. **Invalid action attempts:** attempts made while a required ground-truth precondition is unsatisfied.
3. **Precondition F1:** precision and recall of learned required preconditions against benchmark ground truth.
4. **Effect F1:** precision and recall of learned immediate effects against benchmark ground truth.

12.2 Secondary metrics

- action count per successful task;
- model calls and input/output tokens;
- wall-clock latency by phase;
- experiments used per validated schema;
- schema coverage of held-out tasks;
- contradiction rate;
- calibration of confidence scores;
- knowledge units and serialized memory size;
- success under layout shift;
- fallback-policy frequency;
- unsafe actions proposed, blocked, and executed.

12.3 Experimental procedure

- Use matched task instances across systems.
- Use at least three independent runs per task/configuration when model nondeterminism is present.
- Fix model version, temperature, prompt version, viewport, and action budgets.
- Report the number of tasks and runs explicitly.
- Report mean values and 95% confidence intervals where meaningful.
- Use paired comparisons because systems operate on the same task instances.
- For binary task success, use a paired test such as McNemar's test or a paired bootstrap interval.
- Report failures by category, not only aggregate success.

- Preserve raw run artifacts for audit.

12.4 Interpretation rule

The proposed system is considered empirically supported only when improvements are observed on held-out conditional or compositional tasks, not merely on repeated training trajectories. Efficiency improvements MUST be reported alongside exploration cost.

13. Safety Requirements

13.1 Action taxonomy

Every semantic action MUST be assigned one safety class:

- observational;
- harmless_reversible;
- state_changing_reversible;
- high_impact_or_irreversible.

Active experiments MAY use only the first three classes inside a resettable sandbox. They MUST NOT use high_impact_or_irreversible actions.

13.2 Prohibited autonomous experiments

The experiment selector MUST block actions involving:

- deletion or destructive reset outside the benchmark;
- sending or submitting external communications;
- purchases, payments, checkout, or financial transfer;
- account creation, account deletion, credential changes, or permission changes;
- publishing, deployment, or form submission to external systems;
- file operations outside the configured artifact/sandbox directory;
- system-level changes not covered by deterministic reset.

13.3 Evidence integrity

- The model MUST NOT mark its own hypothesis as proven.
- Only validated observations and evaluator-safe experiment outcomes may change evidence counts.

- Ambiguous outcomes MUST remain ambiguous.
- All experiments MUST be logged before execution with target hypothesis and expected discriminating outcome.
- Credentials and personal data MUST NOT be stored in the action model.

14. Verification and Testing Requirements

14.1 Unit tests

Tests MUST cover:

- predicate validation and normalization;
- position-independent predicate identity;
- predicate delta extraction;
- outcome classification;
- effect support and contradiction updates;
- precondition status transitions;
- confidence calculation;
- schema serialization and migration;
- safe experiment filtering;
- bounded plan search;
- prevention of schema use when required predicates are absent;
- action re-grounding after layout changes;
- atomic action-model export.

14.2 Integration tests

Tests MUST demonstrate:

1. learning an immediate effect from a verified transition;
2. keeping a precondition uncertain after passive correlated evidence;
3. selecting a safe contrasting intervention;
4. confirming a required precondition from controlled success/failure evidence;
5. lowering confidence after contradiction;
6. satisfying a missing precondition before executing a goal action;
7. composing two learned schemas for an unseen goal;
8. completing the same functional task under a layout shift;
9. blocking an unsafe experiment;
10. running every baseline through the same benchmark harness.

14.3 Pixel-only compliance test

The benchmark agent process MUST be instrumented or mocked to fail if it attempts to inspect DOM, accessibility, URL, title, application source, hidden evaluator state, or benchmark APIs. Evaluator access MUST remain in a separate process or interface not passed to the agent.

14.4 Completion gate

Implementation is complete only when:

- all existing regression tests pass;
- all new unit and integration tests pass;
- benchmark reset and scoring are deterministic;
- every baseline is reproducible from a documented command;
- raw evidence can regenerate action-model.json;
- final experiments have been run on the frozen task split;
- results, including negative results, are documented.

15. Development Phases

Phase 0 - Freeze the baseline

Deliverables:

- tag or record the current graph-based implementation;
- preserve the 31-test regression baseline;
- add comparable memory-mode configuration;
- document current graph limitations;
- freeze initial benchmark and metric definitions.

Exit criterion: stateless and graph modes run through the same harness.

Phase 1 - Grounded predicates

Deliverables:

- typed predicate model;
- visual grounding evidence;
- controlled vocabulary for benchmark domains;
- predicate normalization and delta tests.

Exit criterion: predicate states are correctly extracted on the controlled benchmark's development layouts.

Phase 2 - Passive action-effect schemas

Deliverables:

- semantic action identity;
- before/action/after transition ingestion;
- candidate effects;
- evidence store and confidence;
- passive action-effect baseline.

Exit criterion: the system produces inspectable effect schemas and verifies them on repeated transitions.

Phase 3 - Precondition hypotheses

Deliverables:

- success/ineffective contrast analysis;
- required/not-required/conditional/unknown statuses;
- contradiction handling;
- passive explicit-action-model baseline.

Exit criterion: precondition hypotheses are derived from evidence and never directly asserted by the LLM.

Phase 4 - Active intervention

Deliverables:

- hypothesis queue;
- safe experiment selector;
- experiment budget and audit log;
- reset integration;
- intervention-based evidence updates.

Exit criterion: the agent distinguishes at least two competing precondition hypotheses in the controlled benchmark.

Phase 5 - Functional planning

Deliverables:

- goal predicate extraction;
- bounded backward chaining;
- confidence/risk ranking;
- runtime effect verification;
- fallback integration.

Exit criterion: the agent solves a held-out composition task by first establishing a missing precondition.

Phase 6 - Evaluation and paper

Deliverables:

- frozen evaluation configuration;
- complete baseline and ablation runs;
- statistical analysis;
- failure taxonomy;

- real-application case study;
- research paper and reproducibility package.

Exit criterion: another team member can reproduce tables and figures from saved artifacts without undocumented manual steps.

16. How the Proposed Method Is Better - and What Must Be Proven

16.1 Compared with stateless agents

Potential advantage: the agent retains verified functional knowledge instead of re-inferring every action from scratch.

Required proof: fewer model calls or invalid attempts on held-out tasks without sacrificing success.

16.2 Compared with trajectory replay

Potential advantage: schemas can compose parts of different experiences and do not require an identical goal or complete prior trajectory.

Required proof: success on composition tasks for which no identical trajectory exists.

16.3 Compared with screen/action graphs

Potential advantage: preconditions are explicit semantic relations rather than being encoded only by a source screen. This can reduce state explosion and explain why an operation is unavailable.

Required proof: better precondition accuracy, fewer invalid actions, or better layout-shift performance than the current graph.

16.4 Compared with free-text action-effect memory

Potential advantage: the method distinguishes conditions, effects, uncertainty, support, and contradiction in a machine-checkable representation.

Required proof: passive effect descriptions alone perform worse on tasks containing hidden conditions.

16.5 Compared with predictive GUI world models

Potential advantage: the learned model is compact, explicit, inspectable, and can expose unmet preconditions to the planner.

Required proof: interpretable rule accuracy and planning value; the project does not need to outperform large learned world models on every task.

16.6 Honest limitation

The method observes rendered interface behavior, not internal program causality. Even intervention-derived rules are causal only relative to the controlled variables and observable scope. The paper SHOULD use "action-model learning" or "intervention-validated preconditions" more often than the unrestricted term "causal discovery."

17. Risks and Mitigations

Predicate instability

Risk: VLM labels vary across observations. **Mitigation:** controlled vocabularies, normalization, evidence grounding, deterministic benchmark screens, and repeated observations.

Confounded preconditions

Risk: several predicates change together, producing a false required condition. **Mitigation:** prefer minimal contrasting interventions and include deliberate distractor predicates in the benchmark.

State explosion

Risk: content changes create excessive graph nodes. **Mitigation:** separate content values from functional predicates and evaluate semantic schemas independently of screenshot identity.

Experiment cost

Risk: active learning consumes more actions than passive memory. **Mitigation:** fixed experiment budgets and reporting total exploration plus downstream cost.

Unsafe exploration

Risk: the agent tests an irreversible operation. **Mitigation:** explicit action taxonomy, allow-list, sandbox-only active mode, deterministic reset, and hard blocking.

Circular evaluation

Risk: the same model generates predicates and judges whether they are correct. **Mitigation:** hidden evaluator ground truth and independent final-state scoring.

Overclaiming causality

Risk: before/after correlation is described as causal. **Mitigation:** reserve causal language for contrasting interventions; report unresolved confounders and observation limits.

Literature collision

Risk: a newer paper addresses the same intersection. **Mitigation:** repeat the literature search before submission and frame contributions as a tested method and benchmark, not an absolute first.

18. Required Deliverables

1. Updated screenshot-only agent with all four memory modes.
2. Persistent, human-readable action-model.json.
3. Visual Function Lab benchmark with frozen task split and hidden evaluator state.
4. Baseline and ablation runner.
5. Unit and integration test suite.
6. Raw run database and reproducible result-export command.
7. Metrics tables and failure analysis.
8. Real desktop application case study.
9. Final paper centered on active visual precondition discovery.
10. Demonstration showing:

- an action fail because a hidden condition is absent;
- the agent form and test a precondition hypothesis;
- the agent store the validated schema;
- the agent later satisfy the condition automatically for an unseen task;
- the same schema survive a layout change.

19. Paper Blueprint

Proposed title

From Screen Graphs to Action Models: Active Visual Discovery of Hidden GUI Preconditions

Abstract structure

1. Problem: screenshot agents and graph memories record transitions but do not explicitly establish when operations are applicable.
2. Gap: immediate action-effect descriptions and state graphs do not separate required conditions from incidental screen context.
3. Method: screenshot-only grounded predicates, evidence-backed action schemas, safe contrasting interventions, and bounded schema planning.
4. Evaluation: controlled hidden-precondition benchmark, current graph baseline, passive-memory baseline, ablations, and real desktop case study.
5. Result: report measured values only after the frozen evaluation.

Paper sections

1. Introduction
2. Related Work
3. Problem Formulation
4. Method
5. Visual Function Lab Benchmark
6. Experimental Setup
7. Results
8. Ablations and Failure Analysis
9. Limitations, Safety, and Ethics
10. Conclusion

Claimed contributions

Subject to successful implementation, the paper SHOULD claim:

1. an explicit representation for visually grounded GUI action preconditions, effects, evidence, and uncertainty;

2. a safe active intervention strategy for distinguishing precondition hypotheses;
3. schema-based planning for unseen conditional and compositional tasks;
4. a controlled benchmark and evaluation protocol for visual precondition learning;
5. an empirical comparison with stateless, trajectory, graph, and passive action-effect memories.

20. Traceability to Review 1

The original Review 1 goals are retained or revised as follows:

Review 1 objective	Decision in this specification
Pure visual perception	Retained as a mandatory information boundary
API/DOM independence	Retained and strengthened with a compliance test
Semantic UI interaction	Retained; extended into predicates and semantic actions
Persistent UI state graph	Retained as baseline and navigation memory
Autonomous graph construction	Retained opportunistically during tasks; not a novelty claim
Layout-change robustness	Retained as an evaluation condition, not the main contribution
Reusable interface knowledge	Revised from screen topology to evidence-backed action schemas
Scriptless natural-language execution	Retained through fallback VLM policy and schema planning
Broad desktop/web/legacy generalization	Reduced to controlled evaluation plus one desktop case study
Graph memory as research gap	Rejected due to direct prior work

21. References

1. Project Team. *Vision-Based UI State Discovery for API-Independent Software Automation*. B.Tech BCSE497J Review 1 presentation, 22 July 2026.
2. B. Xie et al. "GUI-explorer: Autonomous Exploration and Mining of Transition-aware Knowledge for GUI Agent." ACL 2025.
3. Y. Chai et al. "UI-KOBE: Knowledge-Oriented Behavior Exploration for Lightweight Graph-Guided GUI Agents." 2026.
4. Z. Qin et al. "Executable Agentic Memory for GUI Agent." 2026.
5. H. Zhong et al. "ActionEngine: From Reactive to Programmatic GUI Agents via State Machine Memory." 2026.
6. Y. Cao et al. "MobileDreamer: Generative Sketch World Model for GUI Agent." 2026.
7. Y. Zhang et al. "Don't Act Blindly: Robust GUI Automation via Action-Effect Verification and Self-Correction." 2026.
8. D. Vorvul et al. "Graph-Structured Persistent Memory for Efficient LLM-Based Computer Use Agents." *Axioms*, 2026.
9. A. M. Memon, M. E. Pollack, and M. L. Soffa. "Hierarchical GUI Test Case Generation Using Automated Planning." *IEEE Transactions on Software Engineering*, 2001.
10. T. Xie et al. "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments." 2024.

22. Final Implementation Directive

The team will implement a **vision-only, evidence-driven action-model learner** on top of the existing graph-based GUI agent.

The graph answers:

> *Where has the agent been, and what transition followed an action?*

The new action model must answer:

> *What operation is available, what observable conditions enable it, what effect should it produce, how strong is the evidence, and what safe action can establish a missing condition?*

No feature is complete because it produces plausible prose. It is complete only when its claims are grounded, persisted, contradicted when necessary, used by the planner, and evaluated against the frozen baselines on held-out tasks.